

Student Depression — Predicción de Riesgo

Manual de Instalación y Uso

Versión del proyecto: 0.2.0
Proyecto de especialización en Ciencia de Datos e IA

Nota ética: este proyecto trabaja con un dataset sensible de salud mental. El modelo resultante es una herramienta de análisis estadístico con fines académicos, **no** un instrumento de diagnóstico clínico.

1. Introducción

Este manual explica, paso a paso, cómo instalar y ejecutar el proyecto **Student Depression — Predicción de Riesgo**: un pipeline completo de Machine Learning que cubre carga de datos, análisis exploratorio, modelado, seguimiento de experimentos con MLflow, orquestación con Prefect, y despliegue como API, interfaz web (Streamlit) y contenedor Docker.

Está pensado para quien necesite **levantar el proyecto desde cero** en su propia máquina (Windows, macOS o Linux) y probar cada componente.

2. Requisitos previos

- **Python 3.11 o superior** (el proyecto se desarrolló con 3.14).
- **uv** — gestor de entornos y dependencias (<https://docs.astral.sh/uv/>). No es necesario activar manualmente el entorno virtual: todo se ejecuta con `uv run ...`.
- **Git**, para clonar el repositorio.
- **Docker Desktop** (opcional, solo si se va a construir/correr la imagen de la API).
- Conexión a internet, para la primera instalación de dependencias.

3. Instalación

3.1 Clonar el repositorio

```
git clone https://github.com/miloguz/ProyectoAprendizajeEnNube.git
cd ProyectoAprendizajeEnNube
```

3.2 Instalar las dependencias

`uv` crea el entorno virtual (`.venv`) e instala todas las dependencias declaradas en `pyproject.toml` (exactas, gracias al lockfile `uv.lock`):

```
uv sync
```

3.3 Verificar el dataset

El dataset (*Student Depression Dataset*, Kaggle) debe estar en `data/raw/student_depression.csv`. Si el repositorio ya lo incluye, no hay que hacer nada; si no, se descarga de Kaggle y se coloca en esa ruta exacta.

3.4 Ejecutar las pruebas

Para confirmar que la instalación quedó correcta, correr la suite de tests:

```
uv run pytest
```

Debe terminar mostrando todos los tests en verde ("passed").

4. Uso paso a paso

Cada componente puede levantarse por separado. El orden recomendado para una primera prueba completa es: **MLflow -> Prefect (pipeline) -> API -> Streamlit**. Cada bloque de comandos se ejecuta en su propia terminal.

4.1 Notebooks (EDA y modelado exploratorio)

Opcional, para revisar el análisis exploratorio y el modelado paso a paso:

```
uv run jupyter lab notebooks/01_eda.ipynb
```

Los notebooks numerados (01 a 04) se ejecutan en orden: EDA -> modelado -> tuning -> seguimiento con MLflow.

4.2 MLflow — seguimiento de experimentos

Para generar runs de MLflow con los modelos base y tuneados, y luego ver la interfaz web:

```
uv run python -m src.models.train_mlflow
```

```
uv run mlflow ui --backend-store-uri sqlite:///mlflow.db --default-artifact-root ./mlruns
```

Abrir en el navegador: <http://localhost:5000>

4.3 Orquestación con Prefect (pipeline completo)

El pipeline de orquestación entrena, tunea, evalúa y registra el modelo candidato de punta a punta. Requiere un servidor de Prefect corriendo (dejarlo en una terminal aparte):

```
uv run prefect server start
```

Y, en otra terminal, ejecutar el flow (genera `models/candidate_pipeline.joblib`, el artefacto que sirven la API y Streamlit):

```
uv run python -m src.orchestration.flow
```

Dashboard de Prefect: <http://localhost:4200>

4.4 API de predicción (FastAPI)

Requiere haber generado antes el artefacto del modelo (paso 4.3, o al menos `uv run python -m src.models.train_mlflow`):

```
uv run uvicorn src.api.main:app --reload
```

- Documentación interactiva (Swagger): <http://localhost:8000/docs>
- GET `/health` — estado del servicio y del modelo cargado.
- POST `/predict` — recibe los datos de un estudiante y devuelve la predicción de riesgo.

4.5 Interfaz Streamlit

Con la API ya corriendo (paso 4.4), en otra terminal:

```
uv run streamlit run src/app/streamlit_app.py
```

Se abre en `http://localhost:8501`. Si la API corre en otra URL, se indica con la variable de entorno `STREAMLIT_API_URL`.

4.6 Despliegue con Docker (opcional)

El artefacto del modelo no viaja en la imagen por defecto: hay que generarlo antes (paso 4.3) y luego construir la imagen:

```
docker build -t student-depression-api:0.2.0 -t student-depression-api:latest .  
docker run -p 8000:8000 student-depression-api:0.2.0
```

La API queda disponible igual en `http://localhost:8000`. El contenedor corre con usuario no-root y trae un healthcheck nativo contra `/health`.

5. Estructura del proyecto

```
student-depression-mlops/
  data/raw/                # Dataset original
  notebooks/               # EDA, modelado, tuning, MLflow (01-04)
  src/
    config/                # Constantes y configuracion de MLflow
    data/                  # Carga de datos
    features/              # Feature engineering
    models/                # Entrenamiento, evaluacion, tracking
    orchestration/         # Pipeline orquestado con Prefect
    api/                   # API de prediccion (FastAPI)
    app/                   # Interfaz de prediccion (Streamlit)
  tests/                   # Pruebas unitarias (pytest)
  models/                  # Artefactos entrenados (no en git)
  Dockerfile
  CHANGELOG.md
  pyproject.toml
```

6. Solución de problemas comunes

El comando "uv run" falla o no encuentra el proyecto

Si el proyecto se movió o copió de carpeta después de instalado, el entorno virtual puede quedar con rutas obsoletas. Solución: borrar la carpeta `.venv` y volver a correr `uv sync`.

La API responde 503 o "Modelo no disponible"

Falta el artefacto `models/candidate_pipeline.joblib`. Generarlo con el pipeline de orquestación (paso 4.3) o con `uv run python -m src.models.train_mlflow`.

Streamlit no puede conectarse a la API

Confirmar que la API está corriendo en `http://localhost:8000` (o ajustar `STREAMLIT_API_URL` si corre en otro puerto/host).

Prefect no logra conectarse a su servidor

El flow de orquestación necesita un servidor de Prefect activo. Dejar `uv run prefect server start` corriendo en una terminal aparte antes de ejecutar el flow.

MLflow rechaza el backend de archivos

Si aparece un error de "file store en modo mantenimiento", fijar la variable de entorno antes de lanzar la UI manualmente:

```
$env:MLFLOW_ALLOW_FILE_STORE = "true"    # PowerShell
export MLFLOW_ALLOW_FILE_STORE=true # bash
```

7. Nota ética

Este proyecto usa un dataset con información sensible sobre salud mental. Se emplea exclusivamente con fines académicos. El modelo, la API y la interfaz Streamlit son herramientas de **análisis estadístico**, **no** instrumentos de diagnóstico clínico. Cualquier uso real exigiría validación profesional, consentimiento informado y supervisión humana.